

DEEP LEARNING FOR FINANCE

MODULE 2 | LESSON 2

CONVOLUTIONS FOR IMAGES

Reading Time	50 minutes
Prior Knowledge	Neural Networks, gradient descent, machine learning
Keywords	Convolution, computer vision, deep learning

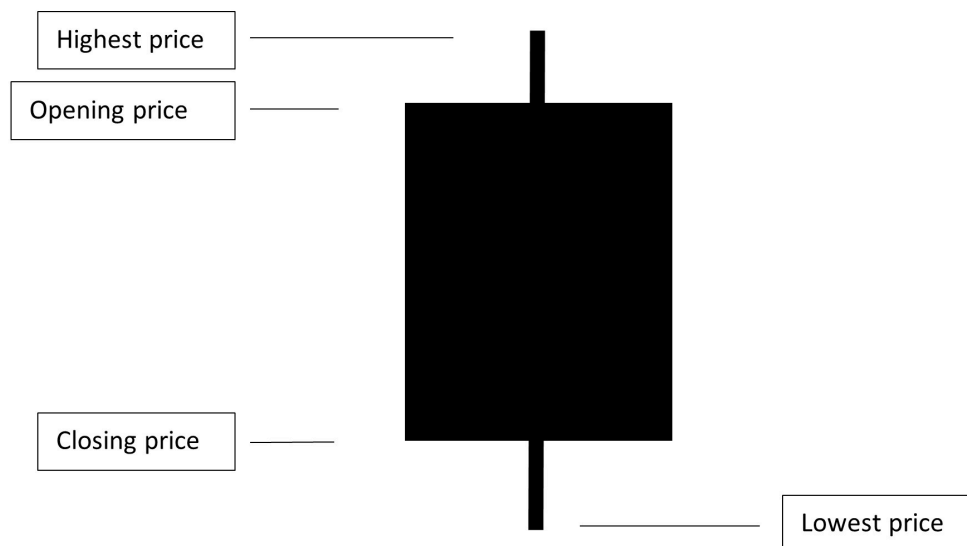
1. CANDLESTICK CHARTS

In lesson 1, we described the key element of a convolutional neural network (in short, CNN or ConvNet) that is the convolution operator that maps an input layer with the structure of an image and generates an output layer that is meant to extract relevant features of the image to predict its class.

Our study and description of ConvNets will be illustrated with a specific application. We are going to study the extent to which patterns shown in candlestick charts of stock prices can predict stock returns by processing the plot with a trained ConvNet.

A candlestick is a type of price chart used in technical analysis that displays the high, low, open, and closing prices of a security for a specific period, usually a trading day. The wide part of the candlestick tells whether the closing price was higher or lower than the opening price (black/red if the stock closed lower, white/green if the stock closed higher). A candlestick's shape varies based on the relationship between the day's high, low, opening, and closing prices. The analysis of candlesticks is suitable for liquid financial assets such as stocks, currencies, and futures.

The figure below illustrates the main elements of a candlestick chart that uses a white/black scheme and depicts a day where a security has displayed a negative return (otherwise, the wide part of the candlestick would be white). Candlestick charting then involves drawing sequences of this type of figure to depict the evolution of the price of a security over time. Usually, additional information is included in these charts, such as trading volume during the period.



Investors that are so-called technical analysts make their entry and exit trades based on observing patterns from candlestick charts that, according to their knowledge, are able to predict the expected returns of securities over the short and medium run. Their main idea is that patterns displayed by stocks over the recent past reflect investors' sentiment about a security. For instance, long white/green candlesticks indicate strong buying pressures, and technical analysts usually look for trends, supports, and bandwidths that guide their forecasts about future prices. This type of technique, also called "chartism," has identified a large array of different shapes displayed by candlestick charts that are claimed to predict future returns: the "hanging man," the "hammer," the "morning star," the "shoulder-head-shoulder," and so on. The technical analysis community is quite large, although it is frequently ignored by the academic finance community, ([here](#) and [here](#) you can find academic papers that provide systematic studies of technical analysis).

Our objective is to evaluate the extent to which we can train a ConvNet that automatically processes the candlestick plots and identifies those patterns that are relevant for the prediction of returns.

2. WORKING WITH CANDLESTICK IMAGES IN PYTHON AND TENSORFLOW-KERAS

Next, to illustrate the type of information we deal with, we are going to read some candlestick charts from our directory. We have generated more than 1,500 charts from the price evolution of two ETFs, SPY and QQQ, over 20 days. Attached to each chart, there is a label that informs about the 25-day cumulative return that follows the pattern in the plot.

With TensorFlow-Keras, we can read and process images. Below, we load an example of a candlestick pattern that we're going to exploit in our applications. We use a green/red color scheme to differentiate between "up" and "down" days. The bottom of the chart also includes information about the trading volume during the day.

```
In [1]: import tensorflow as tf
```

```
2025-05-06 21:12:17.596595: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
2025-05-06 21:12:17.654517: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.
To enable the following instructions: AVX2 AVX512F AVX512_VNNI AVX512_BF16 AVX512_FP16 AVX_VNNI AMX_TILE AMX_INT8 AMX_BF16 FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
2025-05-06 21:12:18.915439: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
```

```
In [2]: img = tf.keras.utils.load_img("QQQ_644.png", keep_aspect_ratio="True")
img
```

Out[2]:



Above, we told TensorFlow to read and process the information directly as an image. As we described in lesson 1, ConvNets process images as tensors. To see the structure of these tensors, we can tell TensorFlow to read the image as an array.

The images have a resolution of 490x490 pixels and have three channels (red, green, and blue). Each pixel is then characterized by a three-dimensional vector, where each element takes values between 1 and 255. For instance, a pixel with values (255,255,255) represents

a fully-white pixel, a pixel with values (0,0,0) is a fully black pixel, while combinations in the middle capture different combinations among red, green, and blue colors.

```
In [3]: img_arr = tf.keras.utils.img_to_array(img)
img_arr.shape
```

```
Out[3]: (490, 490, 3)
```

```
In [4]: img_arr
```

```

Out[4]: array([[255., 255., 255.],
               [255., 255., 255.],
               [255., 255., 255.],
               ...,
               [255., 255., 255.],
               [255., 255., 255.],
               [255., 255., 255.]],

              [[255., 255., 255.],
               [255., 255., 255.],
               [255., 255., 255.],
               ...,
               [255., 255., 255.],
               [255., 255., 255.],
               [255., 255., 255.]],

              [[255., 255., 255.],
               [255., 255., 255.],
               [255., 255., 255.],
               ...,
               [255., 255., 255.],
               [255., 255., 255.],
               [255., 255., 255.]],

              ...,

              [[255., 255., 255.],
               [255., 255., 255.],
               [255., 255., 255.],
               ...,
               [255., 255., 255.],
               [255., 255., 255.],
               [255., 255., 255.]],

              [[255., 255., 255.],
               [255., 255., 255.],
               [255., 255., 255.],
               ...,
               [255., 255., 255.],
               [255., 255., 255.],
               [255., 255., 255.]],

              [[255., 255., 255.],
               [255., 255., 255.],
               [255., 255., 255.],
               ...,
               [255., 255., 255.],
               [255., 255., 255.],
               [255., 255., 255.]], dtype=float32)

```

We can also opt to read images using a black/white color scheme. Notice that with this transformation we cannot determine if the price went up or down during the day (we would need to further transform the images to fill in the "up" days in white).

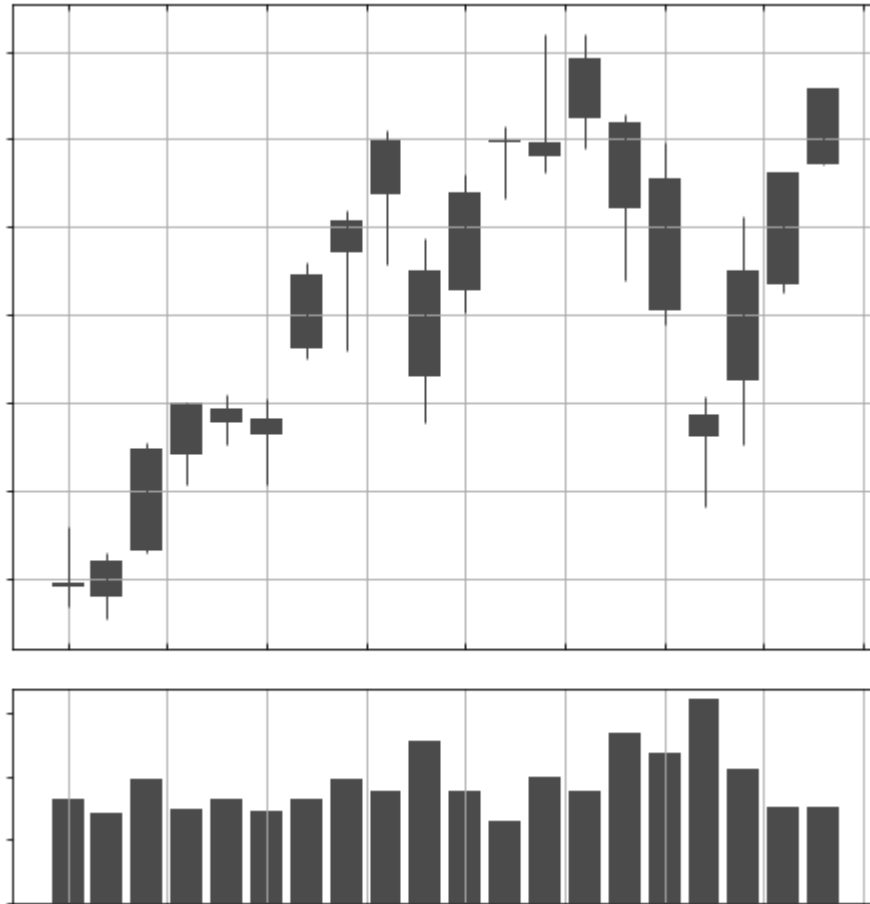
```

In [5]: img_grey = tf.keras.utils.load_img(
          "QQQ_644.png",
          color_mode="grayscale",

```

```
)  
img_grey
```

Out[5]:



With a black/white color scheme, we keep the same resolution, but we obtain a single channel.

```
In [6]: img_arr_grey = tf.keras.utils.img_to_array(img_grey)  
img_arr_grey.shape
```

Out[6]: (490, 490, 1)

3. THE CONVOLUTION OPERATOR

Next, we are going to illustrate and implement the convolution operator in Python. In later lessons, we will exploit tools embedded in TensorFlow-Keras to build convolutional layers, but here, we'll develop them from scratch. For this, we are going to use numpy arrays.

```
In [7]: import numpy as np
```

The block of code below applies a simple convolution operator to a 3×3 array using a 2×2 kernel filter. Before entering the loop, we define the output layer as an array of zeros and fill it with the corresponding values during the iteration. Notice how we slide the kernel over all the elements of the convolution's output.

```
In [8]: # Define input and kernel
input = np.array([[1, 1, 1], [0, 1, 0], [1, 0, 1]])
kern = np.array([[1, 2], [0, 1]])
# Initialize the output array of the convolution
convolution = np.zeros(
    (input.shape[0] - kern.shape[0] + 1, input.shape[1] - kern.shape[1] + 1)
)
for j in range(input.shape[1]): # Loop over width
    if j + kern.shape[1] > input.shape[1]: # Stop when kernel window exceeds wi
        break
    for i in range(input.shape[0]): # Loop over height
        if i + kern.shape[0] > input.shape[0]: # Stop when kernel window excee
            break
        # Apply convolution operator
        convolution[i, j] = np.sum(
            kern * input[i : i + kern.shape[0], j : j + kern.shape[1]]
        )
convolution
```

```
Out[8]: array([[4., 3.],
               [2., 2.]])
```

We can use TensorFlow to make the same computations, as we show below. Please check the online documentation for the function `tf.nn.conv2d()`.

```
In [9]: input = np.array([[1, 1, 1], [0, 1, 0], [1, 0, 1]])
kern = np.array([[1, 2], [0, 1]])
input0 = np.zeros((1, input.shape[0], input.shape[1], 1))
input0[0, :, :, 0] = input
kern0 = np.zeros((2, 2, 1, 1))
kern0[:, :, 0, 0] = np.array([[1, 2], [0, 1]])
conv_tf = tf.nn.conv2d(input0, kern0, 1, "VALID", data_format="NHWC")
conv_tf_arr = tf.keras.utils.img_to_array(conv_tf[0, :, :, 0])
print(conv_tf_arr)
```

```
[[[4.]
   [3.]]

 [[2.]
   [2.]])
```

Below, we define a padding function that adds zeros on all sides to an image. The parameter "padding_width" determines how many columns and rows we are adding on each side. For instance, this means that setting "padding_width=0" means no padding, and "padding_width=1" means adding 2 rows and 2 columns of zeros. The function below is general enough to allow multiple input channels.

```
In [10]: def add_padding(img, padding_width):
# Array of zeros of shape (img + padding_width)
# Check channel dimension
if img.ndim > 2:
    img_with_padding = np.zeros(
        shape=(
            img.shape[0] + padding_width * 2, # We add padding on all sides
            img.shape[1] + padding_width * 2,
            img.shape[2],
        )
    )
```

```

    for channel in range(img.shape[2]):
        # Change the inner elements
        img_with_padding[
            padding_width:-padding_width, padding_width:-padding_width, chan
        ] = img[:, :, channel]
    else:
        img_with_padding = np.zeros(
            shape=(
                img.shape[0] + padding_width * 2, # We add padding on all sides
                img.shape[1] + padding_width * 2,
            )
        )
        # Change the inner elements
        img_with_padding[
            padding_width:-padding_width, padding_width:-padding_width
        ] = img[:, :]
    return img_with_padding

```

The convolution operator then involves making the same operations as above, but for the padded image.

```

In [11]: input = np.array([[1, 1], [0, 1]])
kern = np.array([[1, 2], [0, 1]])
pad = 1
input = add_padding(input, pad)
convolution = np.zeros(
    (input.shape[0] - kern.shape[0] + 1, input.shape[1] - kern.shape[1] + 1)
)
for j in range(input.shape[1]): # Loop over width
    if j + kern.shape[1] > input.shape[1]: # Stop when kernel window exceeds wi
        break
    for i in range(input.shape[0]): # Loop over height
        if i + kern.shape[0] > input.shape[0]: # Stop when kernel window excee
            break
        # Apply convolution operator
        convolution[i, j] = np.sum(
            kern * input[i : i + kern.shape[0], j : j + kern.shape[1]]
        )
print(input)
print(kern)
convolution

```

```

[[0. 0. 0. 0.]
 [0. 1. 1. 0.]
 [0. 0. 1. 0.]
 [0. 0. 0. 0.]]
[[1 2]
 [0 1]]

```

```

Out[11]: array([[1., 1., 0.],
               [2., 4., 1.],
               [0., 2., 1.]])

```

Let's check that we obtain the same result as using TensorFlow in-built function:

```

In [12]: input = np.array([[1, 1], [0, 1]])
kern = np.array([[1, 2], [0, 1]])
input0 = np.zeros((1, input.shape[0], input.shape[1], 1))
input0[0, :, :, 0] = input

```



```
kern0 = np.zeros((2, 2, 1, 1))
kern0[:, :, 0, 0] = kern
conv_tf = tf.nn.conv2d(
    input0, kern0, 1, [[0, 0], [1, 1], [1, 1], [0, 0]], data_format="NHWC"
)
print(conv_tf)
```

```
tf.Tensor(
[[[1.]
  [1.]
  [0.]]

 [[2.]
  [4.]
  [1.]]

 [[0.]
  [2.]
  [1.]]], shape=(1, 3, 3, 1), dtype=float64)
```

Now, we show how to implement stride, the same horizontally and vertically, in the convolution. This means applying the convolution only in those locations (i,j) in the input layer where both i and j are divisible by the stride parameter.

```
In [13]: input = np.array([[1, 1, 1], [1, 0, 1], [0, 1, 0]])
kern = np.array([[1, 2], [0, 1]])
pad = 1
strides = 2
output = [
    np.floor((input.shape[0] - kern.shape[0] + pad + strides) / strides).astype(
        np.floor((input.shape[1] - kern.shape[1] + pad + strides) / strides).astype(
            int
        )
    )
]
if pad != 0:
    input = add_padding(input, pad)
convolution = np.zeros(output)
for j in range(input.shape[1]): # Loop over width
    if j + kern.shape[1] > input.shape[1]: # Stop when kernel window exceeds wi
        break
    if j % strides == 0: # Apply kernel taking into account the stride
        for i in range(input.shape[0]): # Loop over height
            if (
                i + kern.shape[0] > input.shape[0]
            ): # Stop when kernel window exceeds height
                break
            if i % strides == 0: # Apply kernel taking into account the stride
                # Apply convolution operator
                convolution[int(i / strides), int(j / strides)] = np.sum(
                    kern * input[i : i + kern.shape[0], j : j + kern.shape[1]]
                )
convolution
```

```
Out[13]: array([[1., 1.],
                [2., 2.]])
```

And with TensorFlow:

```
In [14]: input = np.array([[1, 1, 1], [1, 0, 1], [0, 1, 0]])
kern = np.array([[1, 2], [0, 1]])
input0 = np.zeros((1, input.shape[0], input.shape[1], 1))
```

```

input0[0, :, :, 0] = input
kern0 = np.zeros((2, 2, 1, 1))
kern0[:, :, 0, 0] = kern
conv_tf = tf.nn.conv2d(
    input0, kern0, 2, [[0, 0], [1, 1], [1, 1], [0, 0]], data_format="NHWC"
)
print(conv_tf)

```

```

tf.Tensor(
[[[[1.]
    [1.]]

  [[2.]
    [2.]]]], shape=(1, 2, 2, 1), dtype=float64)

```

Finally, we augment the above computations, allowing any arbitrary channel dimension for the input as well as the output. As we shall describe in lesson 3, applying convolutions with multiple input channels implies associating one kernel to each input channel. For each output channel, the convolution operator adds up the results of using each of the kernels in each input channel.

Let's omit the details of this computation and just focus on illustrating the outcome of the operation. After reviewing lesson 3, you can return to this code to understand all the computations.

```

In [15]: def convolution_xd(input, kernel, pad=0, stride=1):
          # Determine the size of the output layer
          output = [
              np.floor((input.shape[0] - kernel.shape[0] + 2 * pad + stride) / stride)
              int
          ),
          np.floor((input.shape[1] - kernel.shape[1] + 2 * pad + stride) / stride)
          int
          ),
          kernel.shape[3],
          ]
          # Add padding if necessary
          if pad != 0:
              input = add_padding(input, pad)
          # Initialize the output array of the convolution
          convolution = np.zeros((output[0], output[1], kernel.shape[3]))
          for k in range(kernel.shape[3]): # Loop over output channels
              for j in range(input.shape[1]): # Loop over width
                  if (
                      j + kernel.shape[1] > input.shape[1]
                  ): # Stop when kernel window exceeds width
                      break
                  if j % stride == 0: # Apply kernel taking into account the stride
                      for i in range(input.shape[0]): # Loop over height
                          if (
                              i + kernel.shape[0] > input.shape[0]
                          ): # Stop when kernel window exceeds height
                              break
                          try:
                              if (
                                  i % stride == 0
                              ): # Apply kernel taking into account the stride
                                  for h in range(input.shape[2]): # Loop over input c

```

```

        # Apply convolution operator in output channel k
        convolution[
            int(i / stride), int(j / stride), k
        ] += np.sum(
            kernel[:, :, h, k]
            * input[
                i : i + kernel.shape[0],
                j : j + kernel.shape[1],
                h,
            ]
        )
    except: # noqa E722
        break
return convolution

```

Below, we define as input image the one we loaded from our sample dataset. We want to obtain a three-dimensional output layer, so we must define a kernel of size kernel height $\times$ kernel width $\times$ input channels $\times$ output channels. In a ConvNet, the elements of this kernel will represent (some of) the trainable parameters that we want to use to predict the label of images.

```

In [16]: input = img_arr[..., ::-1]
channels_out = 3
#
kernel = np.zeros((3, 3, input.shape[2], channels_out))
kernel[:, :, 0, 0] = np.array([[ -1, -1, -1], [-1, 10, -1], [-1, -1, -1]])
kernel[:, :, 0, 1] = np.array([[ 1, 1, 1], [ 1, 10, 1], [ 1, 1, 1]])
kernel[:, :, 0, 2] = np.array([[ 1, -1, 1], [ 1, -10, 1], [ 1, -1, 1]])
#
kernel[:, :, 1, 0] = np.array([[ -1, -1, -1], [-1, 10, -1], [-1, -1, -1]])
kernel[:, :, 1, 1] = np.array([[ 1, 1, 1], [ 1, -10, 1], [ 1, 1, 1]])
kernel[:, :, 1, 2] = np.array([[ 1, -1, 1], [ 1, 10, 1], [ 1, -1, 1]])
#
kernel[:, :, 2, 0] = np.array([[ -1, -1, -1], [-1, -10, -1], [-1, -1, -1]])
kernel[:, :, 2, 1] = np.array([[ 1, 1, 1], [ 1, -10, 1], [ 1, 1, 1]])
kernel[:, :, 2, 2] = np.array([[ 1, -1, 1], [ 1, -10, 1], [ 1, -1, 1]])
#
convolution = convolution_xd(input, kernel, 4, 3)
convolution.shape

```

Out[16]: (166, 166, 3)

Below, we show as an array the output layer of this convolution. This is hard to interpret, so below, we also represent as images each of the output layers. Notice that this ad-hoc kernel seems to capture separate information, in each channel, about the "up" moves, "down" moves, and the vertical "edges" of the image.

```

In [17]: convolution

```

```

Out[17]: array([[ 0.,  0.,  0.],
 [ 0.,  0.,  0.],
 [ 0.,  0.,  0.],
 ...,
 [ 0.,  0.,  0.],
 [ 0.,  0.,  0.],
 [ 0.,  0.,  0.]],

 [[ 0.,  0.,  0.],
 [ 255., -255., -1785.],
 [-1275., 1275., -255.],
 ...,
 [-1275., 1275., -255.],
 [ 255., -255., -1785.],
 [ 0.,  0.,  0.]],

 [[ 0.,  0.,  0.],
 [-1275., 1275., -1785.],
 [-3570., 3570.,  510.],
 ...,
 [-3570., 3570.,  510.],
 [-1275., 1275., -1785.],
 [ 0.,  0.,  0.]],

 ...,

 [[ 0.,  0.,  0.],
 [-1275., 1275., -1785.],
 [-3570., 3570.,  510.],
 ...,
 [-3570., 3570.,  510.],
 [-1275., 1275., -1785.],
 [ 0.,  0.,  0.]],

 [[ 0.,  0.,  0.],
 [ 255., -255., -1785.],
 [-1275., 1275., -255.],
 ...,
 [-1275., 1275., -255.],
 [ 255., -255., -1785.],
 [ 0.,  0.,  0.]],

 [[ 0.,  0.,  0.],
 [ 0.,  0.,  0.],
 [ 0.,  0.,  0.],
 ...,
 [ 0.,  0.,  0.],
 [ 0.,  0.,  0.],
 [ 0.,  0.,  0.]])

```

4. CONCLUSION

In this notebook, we have set up the ingredients of our stock return prediction model using ConvNets and candlestick charts. Moreover, we have described the basic tools to read and depict an image in Python and we have implemented the convolution operator from scratch.

Lesson 3 builds on the convolution operator with multiple input and output channels, as well as other useful layers that we can add to ConvNets.

See you there!

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.

Module 2: Lesson 1

Convolutional Neural Networks: Introduction



Outline

- ▶ From fully connected layers to convolutions
- ▶ Convolutions for images
- ▶ Padding and stride

Convolutional neural network

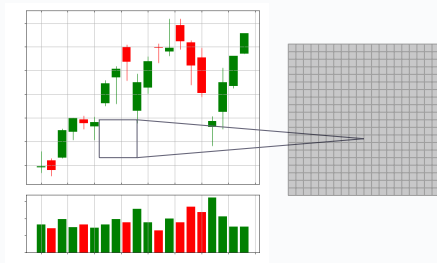
Image data presents a structured source of information that differs from the one-dimensional vectors that we analyze in the context of MLPs.

An image can be represented as a two-dimensional grid of pixels of dimension height $\times$ width.

We can use images as inputs of MLPs by flattening those pixels.

However, we can preserve and exploit the spatial structure of each image's pixels to obtain better models for image recognition.

Convolutional Neural Networks (CNNs) represent a powerful family of neural networks designed for this purpose.



From fully connected layers to convolutions

Consider an MLP that uses two-dimensional images X as inputs, with H representing the immediate hidden representation with the same shape as X .

A fully-connected layer that maps X to H has, at each location (i, j) in H , the following expression:

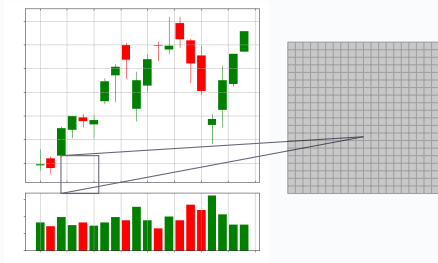
$$[H]_{i,j} = [U]_{i,j} + \sum_a \sum_b [V]_{i,j,a,b} [X]_{i+a,j+b} \quad (1)$$

The hidden representation $[H]_{i,j}$ comes from a bias term $[U]_{i,j}$ plus a summation over all the image's pixels, each weighted by $[V]_{i,j,a,b}$, the parameter of pixel's $(i+a, j+b)$ of input $[X]_{i+a,j+b}$ (V is a fourth-order tensor).

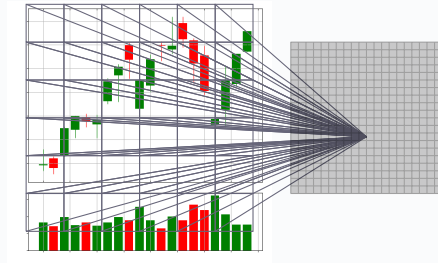
Notice that for each hidden state $[H]_{i,j}$ we would have specific weights that would link pixel (i, j) to all the other pixels in the image. That is, in a 1-megapixel image we would face 10^{12} parameters to estimate (aside from the biases).

Convolutional neural network

Hidden unit, $[H]$



Hidden unit, $[H]$



From fully connected layers to convolutions

To make the problem computationally feasible, we first impose that the biases and weights are the same across pixels that are at the same distance from each hidden representation:

$$[H]_{i,j} = u + \sum_a \sum_b [V]_{a,b} [X]_{i+a,j+b} \quad (2)$$

We are making the weights that lead to the hidden representation $[H]_{i,j}$ independent of the location of the pixel (i,j) .

This assumption is computationally convenient and makes sense if the patterns that can determine the class of an image can be located anywhere in the image.

From fully connected layers to convolutions

To make the problem even more manageable, we only consider that the hidden unit $[H]_{i,j}$ is influenced by pixels in the vicinity of (i,j) :

$$[H]_{i,j} = u + \sum_{a=-\Delta}^{\Delta} \sum_{b=-\Delta}^{\Delta} [V]_{a,b} [X]_{i+a,j+b} \quad (3)$$

where Δ is typically smaller than 10.

This means that we should not look very far away from pixel (i,j) to extract relevant information to identify the patterns of that pixel.

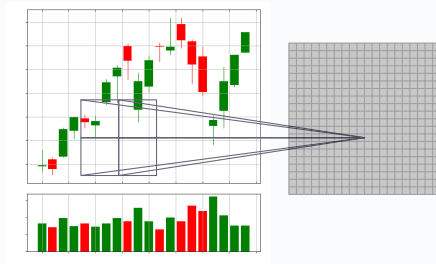
The above expression is what we call a **convolutional layer**.

Convolutional Neural Networks (CNNs) are a special type of neural network that contains convolutional layers, among others, where $[V]_{a,b}$ represents the weights to learn (also called convolutional kernels, or filters).

While the above computations restrict the degree of connections between pixels, the idea is that successive convolutional layers should represent larger and more complex aspects of an image.

Convolutional neural network

Hidden unit, $[H]$



From fully connected layers to convolutions

So far we ignored that images consist of three channels: red, green, and blue. Images are third-order tensors.

- Indeed, modern image files may feature a fourth channel for transparency

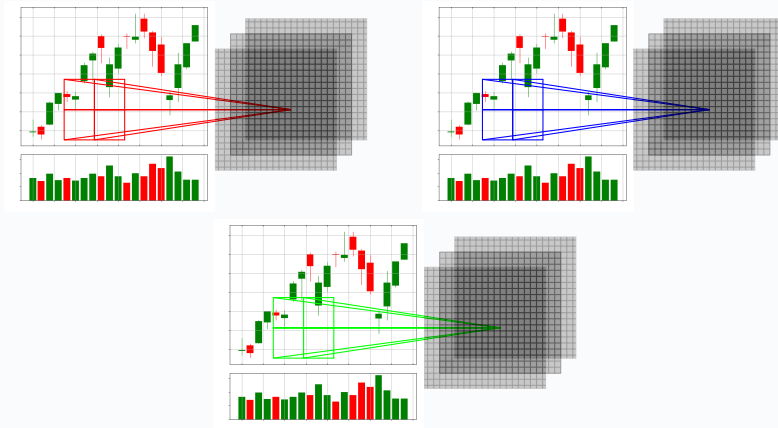
We then need a convolutional filter to have two additional dimensions, $[V]_{a,b,c,d}$, considering that the hidden representation can also have several channels:

$$[H]_{i,j,d} = u_d + \sum_{a=-\Delta}^{\Delta} \sum_{b=-\Delta}^{\Delta} \sum_c [V]_{a,b,c,d} [X]_{i+a,j+b,c} \quad (4)$$

where c indexes the input channels and d indexes the output channels.

While we have so far omitted this, the hidden representation H is usually obtained after the application of an activation function $\phi(\cdot)$ to the outcome of the convolution.

Convolutional neural network



Convolutions for images

Let's see how convolutions work with simple two-dimensional data and hidden representations with a single channel.

INPUT				KERNEL			OUTPUT	
1	2	4		1	2	=	?	?
3	7	2	*	0	1		?	?
2	0	2						

The input tensor has a shape 3×3 , the kernel has a shape or *kernel window* of 2×2 .

We can only properly compute the convolution in locations where the kernel fits wholly in the image so that the output size is

$$(input\ height - kernel\ height + 1) \times (input\ width - kernel\ width + 1) \quad (5)$$

Convolutions for images

Let's see how convolutions work with simple two-dimensional data and hidden representations with a single channel.

$$\begin{array}{c} \text{INPUT} \\ \begin{array}{|c|c|c|} \hline 1 & 2 & 4 \\ \hline 3 & 7 & 2 \\ \hline 2 & 0 & 2 \\ \hline \end{array} \end{array} * \begin{array}{c} \text{KERNEL} \\ \begin{array}{|c|c|} \hline 1 & 2 \\ \hline 0 & 1 \\ \hline \end{array} \end{array} = \begin{array}{c} \text{OUTPUT} \\ \begin{array}{|c|c|} \hline 12 & ? \\ \hline ? & ? \\ \hline \end{array} \end{array}$$

To perform the convolution in an image, we begin with the window positioned at the upper-left corner of the input tensor and slide it both from left to right and top to bottom.

$$\begin{array}{c} \text{INPUT} \\ \begin{array}{|c|c|c|} \hline 1 & 2 & 4 \\ \hline 3 & 7 & 2 \\ \hline 2 & 0 & 2 \\ \hline \end{array} \end{array} * \begin{array}{c} \text{KERNEL} \\ \begin{array}{|c|c|} \hline 1 & 2 \\ \hline 0 & 1 \\ \hline \end{array} \end{array} = \begin{array}{c} \text{OUTPUT} \\ \begin{array}{|c|c|} \hline 12 & 12 \\ \hline 17 & 13 \\ \hline \end{array} \end{array}$$

Warning: Notice that the operator $*$ here represents some abuse of notation, since it refers to element-by-element multiplication and then addition of all terms.

Padding and stride

Convolutional kernels must “fit” into the image, which may lead successive convolutions to “ignore” the boundaries of images.

Several tools allow us to control the size of the output layer and override this problem.

Padding: Consists of adding extra pixels with a value of zero around the boundary of the input image.

INPUT						KERNEL			OUTPUT			
0	0	0	0	0					1	2	4	0
0	1	2	4	0					5	12	12	4
0	3	7	2	0	*	1	2	=	8	17	13	2
0	2	0	2	0		0	1		4	2	4	2
0	0	0	0	0								

With padding, the output size of a convolutional layer is given by:

$$(input\ height - kernel\ height + rows\ of\ padding + 1) \times (input\ width - kernel\ width + columns\ of\ padding + 1)$$

Padding and stride

Stride: Instead of covering the whole input with the kernel in each convolution, we move more than one element at a time.

We may be interested in doing this to reduce computational costs or to downsample.

Above, we used a stride of 1. If we use a stride of 3 horizontally and 2 vertically we get the following:

INPUT						KERNEL			OUTPUT	
0	0	0	0	0						
0	1	2	4	0		1	2		1	0
0	3	7	2	0	*	0	1	=	8	2
0	2	0	2	0						
0	0	0	0	0						

With stride, the output size of a convolutional layer is given by ($\lfloor \cdot \rfloor$ indicates the “floor” operator):

$$\begin{aligned} & \lfloor (input\ height - kernel\ height + rows\ of\ padding + rows\ of\ stride) / rows\ of\ stride \rfloor \\ & \times \\ & \lfloor (input\ width - kernel\ width + columns\ of\ padding + columns\ of\ stride) / columns\ of\ stride \rfloor \end{aligned}$$

Padding and stride

Padding can increase the height and width of the output layer, which is useful to avoid undesirable shrinkage.

Padding also ensures that all pixels are used with the same frequency.

Typically, we opt for symmetric padding on both sides of the input height and width, so that we can refer to padding with a single parameter $p = \text{rows of padding} = \text{columns of padding}$. By default, $p = 0$.

Similarly, when horizontal and vertical strides match, we simply talk about stride s , where by default $s = 1$.

If we set $\text{rows of padding} = \text{kernel height} - 1$, $\text{columns of padding} = \text{kernel width} - 1$, and input's height and width are divisible by the strides on height and width, then the output shape will be

$$(\text{input height} / \text{rows of stride}) \times (\text{input width} / \text{columns of stride})$$

Summary of Lesson 1

In Lesson 1, we have looked at:

- ▶ The concept of convolutional layer in association with image-recognition using neural networks.
- ▶ Padding and stride operations to preserve the dimensionality of input images.

⇒ **References for this lesson:**

Chapter 7 in [Zhang, Aston, et al. "Dive into deep learning." \(2022\).](#)

TO DO NEXT: This lesson does not have an associated Jupyter Notebook. Please advance to the next lesson.

In the next lesson, we will describe the basics of image processing and implement the convolution operator in Python for candlestick charts.

Module 2: Lesson 2

Convolutional Neural Networks: Candlesticks and convolutions



Outline

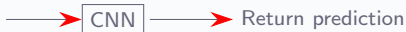
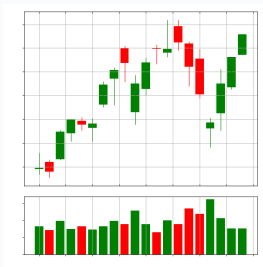
- ▶ Candlestick charts
- ▶ Convolutions for (actual) images

ConvNets for return prediction

In Lesson 1, we described the convolution operator from a fully-connected layer that is constrained to have a limited set of constant weights across hidden representations.

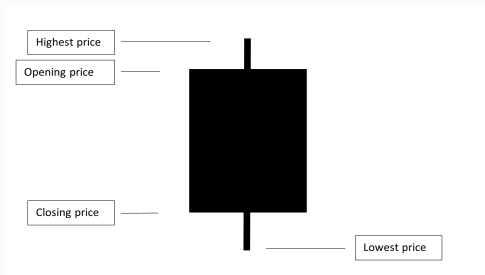
We are going to illustrate the implementation of ConvNets in the context of a problem to predict future stock returns using images.

Our inputs will be the evolution of past returns as depicted in candlestick plots (or charts).

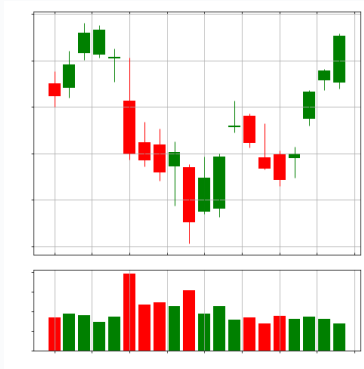


Candlesticks

A candlestick is a type of price chart used in technical analysis that displays the high, low, open, and closing prices of a security for a specific period, usually a trading day.



Processing images in Python

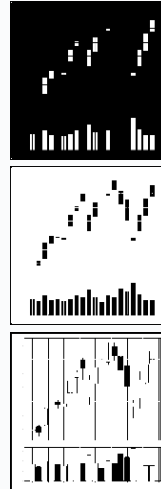
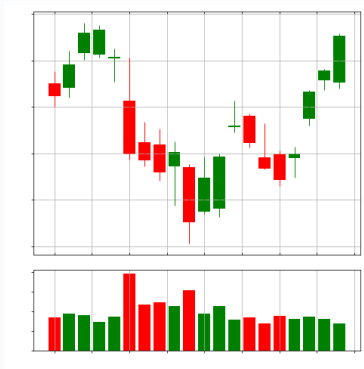


Convolutions from scratch in Python

```
input = np.array([[1, 1, 1], [0, 1, 0], [1, 0, 1]])
kern = np.array([[1, 2], [0, 1]])
convolution = np.zeros((input.shape[0]-kern.shape[0]+1, input.shape[1]-
kern.shape[1]+1))
for i in range(input.shape[0]):
    if i + kern.shape[0] > input.shape[0]:
        break
    for j in range(input.shape[1]):
        if j + kern.shape[1] > input.shape[1]:
            break
        else:
            convolution[i,j] = np.sum(kern * input[i:i + kern.shape[0],j:j + kern.shape[1]])
convolution
```

$$\begin{array}{|c|c|c|} \hline \text{INPUT} & & \\ \hline 1 & 1 & 1 \\ \hline 0 & 1 & 0 \\ \hline 1 & 0 & 1 \\ \hline \end{array} * \begin{array}{|c|c|} \hline \text{KERNEL} & \\ \hline 1 & 2 \\ \hline 0 & 1 \\ \hline \end{array} = \begin{array}{|c|c|} \hline \text{OUTPUT} & \\ \hline 4 & 3 \\ \hline 2 & 2 \\ \hline \end{array}$$

Convolution of a candlestick plot



Summary of Lesson 2

In Lesson 2, we have:

- ▶ Described our stock return prediction problem using ConvNets and candlestick charts
- ▶ Implemented the convolution operator from scratch in Python

⇒ **References for this lesson:**

Chapter 7 in [Zhang, Aston, et al. "Dive into deep learning." \(2022\).](#)

TO DO NEXT: Please, go to the Jupyter Notebook associated with Lesson 2.

In the next lesson, we will describe the convolution operator with multiple input and output channels, as well as additional types of layers that we can include in CNNs to enhance their performance.

Module 2: Lesson 3

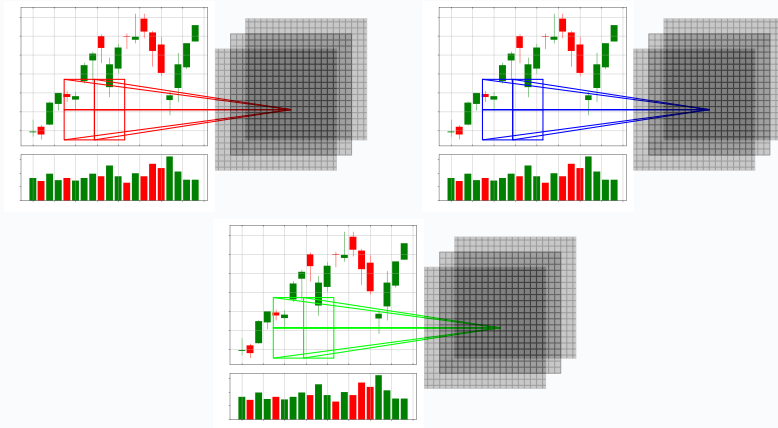
Convolutional Neural Networks: Towards Modern CNNs



Outline

- ▶ Multiple input and output channels
- ▶ Pooling
- ▶ Batch Normalization

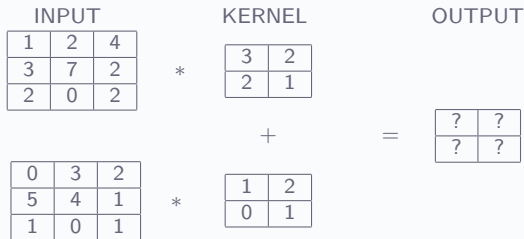
Multiple input channels



Multiple input channels

When the input data contains multiple channels, we need to construct a convolution kernel with the same number of input channels as the input data.

That is, we need a kernel that contains a tensor of shape $\text{kernel height} \times \text{kernel width}$ for every input channel.



Multiple input channels

To obtain the output channel we just add up the results of the convolution for each separate input channel.

INPUT				KERNEL			OUTPUT	
1	2	4		3	2			
3	7	2	*	2	1			
2	0	2						
						+		
0	3	2		1	2			
5	4	1	*	0	1			
1	0	1				=		
							30 ?	
							? ?	

Multiple input channels

To obtain the output channel we just add up the results of the convolution for each separate input channel.

INPUT				KERNEL			OUTPUT	
1	2	4		3	2			
3	7	2	*	2	1			
2	0	2						
					+			
0	3	2		1	2	=	30 38	
5	4	1	*	0	1		40 34	
1	0	1						

Remember that in practice we also need to add a bias term associated with each kernel.

Multiple output channels

When training CNNs, it turns out to be essential to have multiple channels and increase the channel dimension at each layer.

The intuition is that each channel will detect a different set of features when training the network.

For each output channel, we create a kernel tensor of shape kernel height $\times$ kernel width $\times$ input channels.

INPUT		KERNEL Channel 1		OUTPUT Channel 1													
<table><tr><td>1</td><td>2</td><td>4</td></tr><tr><td>3</td><td>7</td><td>2</td></tr><tr><td>2</td><td>0</td><td>2</td></tr></table>	1	2	4	3	7	2	2	0	2	*	<table><tr><td>3</td><td>2</td></tr><tr><td>2</td><td>1</td></tr></table>	3	2	2	1		
1	2	4															
3	7	2															
2	0	2															
3	2																
2	1																
		+															
			=	<table><tr><td>30</td><td>38</td></tr><tr><td>40</td><td>34</td></tr></table>	30	38	40	34									
30	38																
40	34																
<table><tr><td>0</td><td>3</td><td>2</td></tr><tr><td>5</td><td>4</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr></table>	0	3	2	5	4	1	1	0	1	*	<table><tr><td>1</td><td>2</td></tr><tr><td>0</td><td>1</td></tr></table>	1	2	0	1		
0	3	2															
5	4	1															
1	0	1															
1	2																
0	1																

INPUT		KERNEL Channel 2		OUTPUT Channel 2													
<table><tr><td>1</td><td>2</td><td>4</td></tr><tr><td>3</td><td>7</td><td>2</td></tr><tr><td>2</td><td>0</td><td>2</td></tr></table>	1	2	4	3	7	2	2	0	2	*	<table><tr><td>0</td><td>2</td></tr><tr><td>1</td><td>1</td></tr></table>	0	2	1	1		
1	2	4															
3	7	2															
2	0	2															
0	2																
1	1																
		+															
			=	<table><tr><td>27</td><td>30</td></tr><tr><td>27</td><td>11</td></tr></table>	27	30	27	11									
27	30																
27	11																
<table><tr><td>0</td><td>3</td><td>2</td></tr><tr><td>5</td><td>4</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr></table>	0	3	2	5	4	1	1	0	1	*	<table><tr><td>1</td><td>1</td></tr><tr><td>2</td><td>0</td></tr></table>	1	1	2	0		
0	3	2															
5	4	1															
1	0	1															
1	1																
2	0																

The 1×1 convolutional layer

1×1 convolutions are sometimes included in the designs of complex deep networks. These convolutions imply a kernel of dimension 1×1 .

In a single input and output channel framework, the 1×1 convolution only maps the input image to an identical output layer.

1×1 convolutions only work through the channel dimension. This type of convolution leads to output layers with the same height and width as the input.

1×1 convolutions generate linear combinations of elements at the same position across the input channels. However, layers are usually followed by non-linear activations, so that these type of convolutions are not subsumed into other convolutions with kernels of higher dimension.

This type of convolution loses the ability to recognize patterns of interactions among adjacent pixels, but aims at detecting the relevance of specific pixels.

Pooling

In most cases, our ultimate objective about image recognition is to obtain a prediction about the content of a given image. For instance, does a particular candlestick pattern predict a positive or a negative return?

For this, the final layer in a CNN should be sensitive to the entire input image.

Another issue with CNNs is that the output may be sensitive to minor changes in the location of features within images.

Pooling layers serve the dual purpose of downsampling representations while mitigating the sensitivity of convolutional layers to location.

Pooling operators work as convolutions by applying an operation with a fixed window that travels over the input image but, different from convolutions, pooling layers contain no parameters. Besides, pooling leaves unchanged the channel dimension.

The most common Pooling layer is max-pooling (the alternative option is average-pooling):

INPUT		KERNEL		OUTPUT															
<table border="1"><tr><td>1</td><td>2</td><td>5</td></tr><tr><td>3</td><td>4</td><td>2</td></tr><tr><td>6</td><td>1</td><td>3</td></tr></table>	1	2	5	3	4	2	6	1	3		<table border="1"><tr><td colspan="2">2×2 Pooling Layer</td></tr></table>	2×2 Pooling Layer		=	<table border="1"><tr><td>4</td><td>5</td></tr><tr><td>6</td><td>4</td></tr></table>	4	5	6	4
1	2	5																	
3	4	2																	
6	1	3																	
2×2 Pooling Layer																			
4	5																		
6	4																		

Pooling with multiple channels

When processing input layers with multiple channels, the pooling layer treats each input channel separately.

Thus, the number of output channels in a pooling layer is the same as the number of input channels.

Bear in mind also that we can change the pooling window as well as include padding and stride to any pooling layer.



Batch normalization

Making neural networks converge is one of the biggest challenges in the design of architectures: **Batch Normalization** is a popular technique that consistently accelerates convergence.

Batch normalization can be understood in terms of the variable rescaling that we use in data pre-processing.

- ▶ In each training iteration, we first normalize the inputs by subtracting their mean and their standard deviation within the current minibatch.
- ▶ Next, we apply a linear transformation (slope + intercept).

With batch normalization, the choice of batch size becomes even more significant than in its absence

- ▶ Batch normalization works best for moderate minibatches (50 to 100 instances).

Batch normalization adds sufficient noise to training to improve the convergence speed and reduce overfitting, as in dropout.

This tool also reduces the variation in the values of the hidden units that may hamper convergence.

Batch normalization: Details

Denote by $\mathbf{x} \in \mathcal{B}$ an input to Batch Normalization BN from minibatch $\mathcal{B}$. $\mathbf{x}$ has a dimension equal to the size (number of instances) of the minibatch.

Batch normalization transforms $\mathbf{x}$ according to the following expression:

$$BN(\mathbf{x}) = \gamma \odot \frac{\mathbf{x} - \hat{\mu}_{\mathcal{B}}}{\hat{\sigma}_{\mathcal{B}}} + \beta$$

$$\hat{\mu}_{\mathcal{B}} = \frac{1}{|\mathcal{B}|} \sum_{\mathbf{x} \in \mathcal{B}} \mathbf{x}$$

$$\hat{\sigma}_{\mathcal{B}}^2 = \frac{1}{|\mathcal{B}|} \sum_{\mathbf{x} \in \mathcal{B}} (\mathbf{x} - \hat{\mu}_{\mathcal{B}})^2$$

where $\hat{\mu}_{\mathcal{B}}$ is the sample mean and $\hat{\sigma}_{\mathcal{B}}$ the sample standard deviation of the minibatch $\mathcal{B}$, and $\odot$ refers to element-by-element multiplication.

γ and β are trainable scale and shift parameters with the same shape as $\mathbf{x}$.

Batch normalization: Details

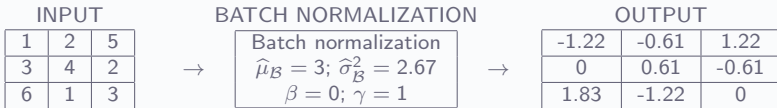
In fully-connected layers of MLPs, if we have a weight parameter $\mathbf{W}$, bias $\mathbf{b}$, and activation function ϕ , an output layer $\mathbf{h}$ with batch normalization is computed as follows:

$$\mathbf{h} = \phi(BN(\mathbf{W}\mathbf{x} + \mathbf{b}))$$

where the mean and variance are computed over the “feature” dimension.

In CNNs, we apply batch normalization on a per-channel basis across all locations.

- ▶ We compute the mean and variance of an input channel across all locations and examples.
- ▶ We apply the same mean and variance to normalize within a given channel so that each channel has one scale parameter and one shift parameter.



Summary of Lesson 3

In Lesson 3, we have looked at:

- ▶ The role of input and output channels in convolutions.
- ▶ The pooling operator.
- ▶ Batch normalization.

⇒ **References for this lesson:**

Chapters 7 & 8 in [Zhang, Aston, et al. "Dive into deep learning." \(2022\).](#)

TO DO NEXT: This lesson does not have an associated Jupyter Notebook. Please advance to the next lesson.

In the next lesson, we will implement Deep CNNs using Tensorflow-Keras in our candlesticks application.

Module 2: Lesson 4

Convolutional Neural Networks: Training a CNN



Outline

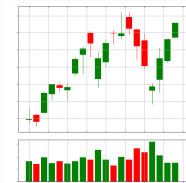
- ▶ Stock return prediction with candlestick charts
- ▶ Network architecture
- ▶ Training the CNN
- ▶ Performance of the model

Stock return prediction with CNNs and candlestick charts

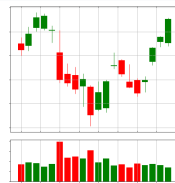
In this last lesson of Module 2, we build and train a fully-fledged, although basic, Convolutional Neural Network.

The objective is to feed the network with images of candlestick patterns, train the architecture, and evaluate its prediction power in the training test.

We exploit a training sample of 1,280 images and a test sample of 360 images that capture candlestick charts of SPY and QQQ.

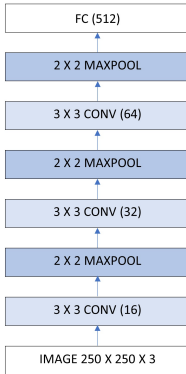


————— "Up" move
—— [CNN] ——— "Up" or "Down" ?



————— "Down" move
—— [CNN] ——— "Up" or "Down" ?

Architecture



The Network architecture shares similarities with AlexNet, by stacking several layers of convolutions and max-pooling.

We increase the width of the layers, i.e., we add channels, as we increase the depth of the network.

After three layers of convolutions + max-pooling, we aggregate all the inputs into a fully connected layer with 512 hidden representations, from where we obtain the last prediction.

Feel free to explore other CNN architectures and check the different prediction abilities of alternative models!

Architecture (some code)

Building a CNN with TensorFlow-Keras turns out to be quite simple.

We keep on adding layers to our network until reaching a flattening layer to introduce (may several) fully-connected layers as in an MLP that allow compiling all the information gathered through the layers to make a prediction.

```
model = tf.keras.Sequential()
# Input layer
model.add(tf.keras.layers.InputLayer(input_shape=(250, 250, 3)))
# Rescaling
model.add(tf.keras.layers.experimental.preprocessing.Rescaling(1./255))
# 1st Convolutional + Max-Pooling layer
model.add(tf.keras.layers.Conv2D(16, 3, activation='relu'))
model.add(tf.keras.layers.MaxPooling2D(2))
# Add more layers in the middle
# nth Convolutional + Max-Pooling layer
model.add(tf.keras.layers.Conv2D(64, 3, activation='relu'))
model.add(tf.keras.layers.MaxPooling2D(2))
# Fully-connected layer
model.add(tf.keras.layers.Flatten())
model.add(tf.keras.layers.Dense(512, activation='relu'))
# Output layer
model.add(tf.keras.layers.Dense(1, activation='sigmoid'))
```


Architecture (adding batch normalization)

We can add batch normalization by inserting layers between the convolution and the activation.

In TensorFlow-Keras, Batch normalization includes two non-trainable parameters, called "momentum" parameters, that smooth the estimation of the current minibatch's mean and variance, using the previous estimate.

Besides, we add a small value to the denominator in batch normalization, 0.001 by default, to avoid division by zero.

```
model.add(tf.keras.layers.Conv2D(16, 5))  
model.add(tf.keras.layers.BatchNormalization())  
model.add(tf.keras.layers.Activation('relu'))
```

Training and performance

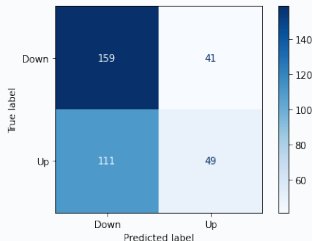
We train the parameters using a standard objective function in binary classification and the RMSprop algorithm.

- We overweight the category with lower frequency ("down") to compensate the lack of balance.

We do not include hyperparameter tuning, so validation and test sets are the same.

- Training is made over 20 epochs and we keep the best model according to its accuracy in validation.
- We see from the optimization how the network begins to overfit without much improvement in validation.

The trained model has some slight predictive ability, although it predicts that prices go "down" most of the time, while in many cases the price goes "up."



Summary of Lesson 4

In Lesson 4, we have:

- ▶ Built a CNN to predict stock prices based on image information from candlestick charts.
- ▶ Trained and evaluated the model's prediction performance.

⇒ **References for this Lesson:**

Chapters 7 & 8 in [Zhang, Aston, et al. "Dive into deep learning." \(2022\).](#)

TO DO NEXT: Please, go to the Jupyter Notebook associated with Lesson 4.

With this, we finalize our introduction to Convolutional Neural Networks. Module 3 will describe further methods meant to improve the convergence and prediction ability of CNNs.